MACHINE LEARNING · LECTURE NOTES

Ensemble Learning

Bias–Variance Trade-off · Bagging · Random Forests · Boosting (AdaBoost, Gradient Boosting, XGBoost)

Contents

1. The Bias–Variance Trade-off
2. Ensemble Learning: The Big Picture
3. Bagging (Bootstrap Aggregating)
4. Random Forests
5. Boosting: AdaBoost, Gradient Boosting, XGBoost

1. The Bias–Variance Trade-off

1.1 Why models make errors

A trained model's prediction error on new data comes from three sources: **bias**, **variance** and **irreducible error**. Our goal in supervised learning is to build a model that predicts well on *unseen* data, so we must understand how each source behaves as the model becomes more or less complex.

1.2 Bias

Bias is the difference between the model's *average* prediction (over many possible training sets) and the true value. It is a systematic error caused by wrong or over-simple assumptions about the data.

- **High bias** – the model is too simple to capture the real pattern. Result: **underfitting**; high error on both training and test data.
- **Low bias** – the model is flexible enough to capture the pattern. Low training error, but if the model is too flexible it also fits noise (see variance).

Example. Fitting a straight line (linear regression) to clearly curved data gives high bias. Linear and logistic regression are typical high-bias models; deep decision trees, k-nearest neighbours with small k, and SVMs with flexible kernels are typical low-bias models.

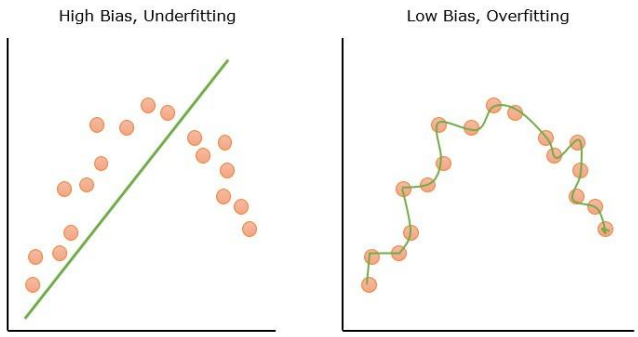


Figure 1. High bias (underfitting) vs. low bias (overfitting) on the same data.

1.3 Variance

Variance measures how much the model's predictions change when the training set changes slightly. A high-variance model reacts to random noise in the training sample as if it were real structure.

- **High variance** – very complex model; fits the training data (including noise) almost perfectly but does poorly on new data. Result: **overfitting**; low training error, high test error.
- **Low variance** – predictions are stable across training sets. Very simple models (or very large training sets) behave this way.

Example. A fully grown (unpruned) decision tree can memorise the training set. Change a few training points and the tree structure changes a lot – that is high variance.

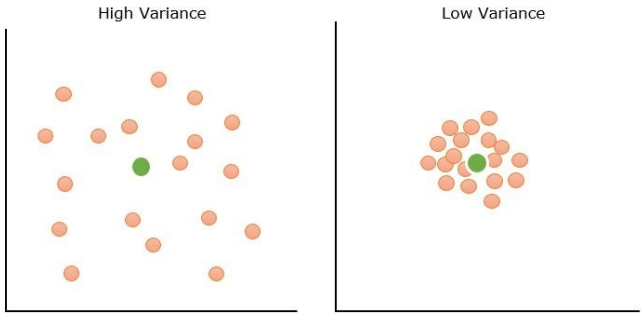


Figure 2. High variance (predictions scattered) vs. low variance (predictions tightly grouped).

1.4 The error decomposition

For squared-error loss, the expected prediction error at a point **x** decomposes exactly as:

$$\mathrm{E}\left[(y - \hat{f}(x))^2\right] = \mathrm{Bias}\left[\hat{f}(x)\right]^2 + \mathrm{Var}\left[\hat{f}(x)\right] + \sigma^2$$

- **Bias²** – how far the average model is from the truth.
- **Variance** – how much individual models scatter around that average.
- **σ² (irreducible error)** – noise in the data itself; no model can remove it.

1.5 The trade-off

As model complexity increases, bias falls but variance rises; as complexity decreases the opposite happens. Total error is U-shaped, and the best model sits near the bottom of the U – the *optimal* region in Figure 3. Lowering one component usually raises the other, which is why it is called a trade-off.

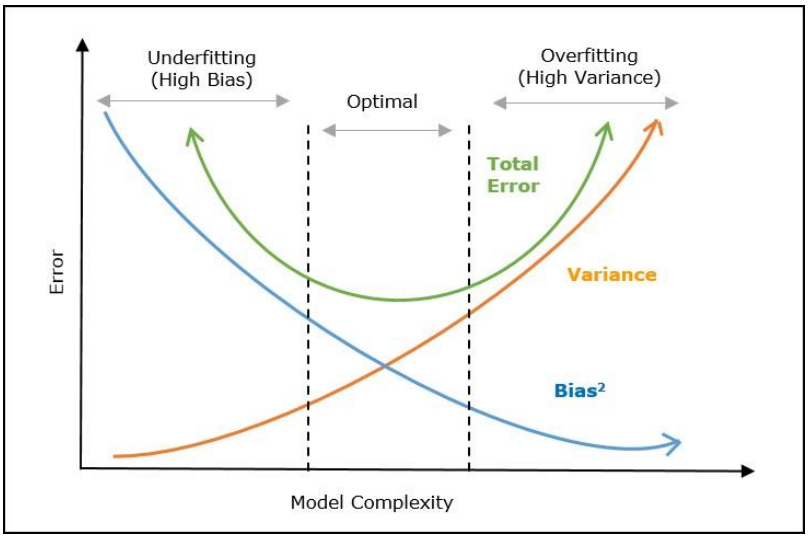


Figure 3. Bias², variance and total error versus model complexity.

1.6 Diagnosing the problem

Situation	Training error	Test error	Diagnosis
Both errors high	High	High	High bias (underfit)
Large gap between errors	Low	High	High variance (overfit)
Both low, small gap	Low	Low	Good fit

1.7 How to reduce each problem

To reduce high bias	To reduce high variance
Use a more complex model	Use a simpler model or prune the tree
Add more (informative) features	Apply regularisation (L1/L2, depth limits)
Reduce regularisation	Collect more training data
Train longer / boost the model	Use cross-validation to detect overfitting early
Ensemble idea: boosting	**Ensemble idea: bagging / random forests**

1.8 Typical bias and variance of common models

Model	Bias	Variance
Linear / logistic regression	High	Low
Decision stump (depth-1 tree)	High	Low
Fully grown decision tree	Low	High
k-nearest neighbours, small k	Low	High
k-nearest neighbours, large k	Higher	Lower
SVM (flexible kernel, large C)	Low	Higher

This table explains the design of the two ensemble families: **deep trees** (low bias, high variance) are the ideal inputs for bagging, while **stumps** (high bias, low variance) are the ideal inputs for boosting.

> **Key idea for the rest of these notes**
>
> Ensembles attack the two error sources differently. **Bagging** averages many high-variance models to *reduce variance*. **Boosting** adds many weak, high-bias models one after another to *reduce bias*.

2. Ensemble Learning:

An **ensemble** combines several models (called *base learners*) so that the combined model performs better, on average, than any individual one. Two broad families are covered in this syllabus:

Family	How models are trained	Main effect	Examples
Bagging	In parallel, on different bootstrap samples; predictions are voted or averaged	Reduces variance	Bagged trees, Random Forest, ExtraTrees
Boosting	Sequentially; each model focuses on the mistakes of the previous ones	Reduces bias (and some variance)	AdaBoost, Gradient Boosting, XGBoost

2.1 Why does voting help?

Suppose each of n classifiers is correct with probability $p = 0.7$ and their errors are **independent**. The majority vote is correct whenever more than half of them are correct (a binomial tail probability):

Number of classifiers	Individual accuracy	Majority-vote accuracy
1	0.70	0.700
3	0.70	0.784
11	0.70	0.922
25	0.70	0.983

Two conditions are essential: each base learner must be **better than random guessing** ($p > 0.5$), and the learners must be **diverse** (their errors must not be strongly correlated). With $p = 0.55$ and 25 classifiers, accuracy only rises to about 0.69; if all classifiers make the same mistakes, voting gives nothing at all. Every ensemble method below is, at heart, a recipe for creating diverse, reasonably accurate base learners.

2.2 Combining predictions

- **Classification:** majority (hard) vote, or average of predicted class probabilities (soft vote).
- **Regression:** simple or weighted average of predictions.
- **Boosting** uses a *weighted* vote: better learners get larger weights.

2.3 Other ensemble ideas (for awareness)

- **Voting ensembles** combine *different* algorithms (e.g. logistic regression, SVM, tree) by hard or soft vote; bagging instead reuses *one* algorithm on different data.
- **Stacking** trains a second-level model (meta-learner) on the predictions of several base models. It is not part of this syllabus' core, but it is the third classic ensemble family.

3. Bagging (Bootstrap Aggregating)

Bagging, introduced by Leo Breiman (1996), trains the **same** learning algorithm (usually a decision tree) on many different **bootstrap samples** of the training set and then aggregates the predictions. It works best for *unstable* learners – models whose predictions change a lot when the training data change slightly, i.e. high-variance models such as unpruned trees.

3.1 Bootstrap sampling

A bootstrap sample of size *m* is drawn from the *m* training examples **uniformly, with replacement**. Some examples appear several times; others do not appear at all.

The probability that a particular example is *not* chosen in one draw is 1 − 1/m, so after *m* draws:

$$P(\text{not chosen}) = \left(1 - \frac{1}{m}\right)^m \;\to\; \frac{1}{e} \approx 0.368 \quad (m \to \infty)$$

$$P(\text{chosen}) = 1 - \left(1 - \frac{1}{m}\right)^m \approx 0.632$$

So each bootstrap sample contains only about **63.2 %** of the distinct training examples; the remaining ≈ 36.8 % are the **out-of-bag (OOB)** examples for that tree.

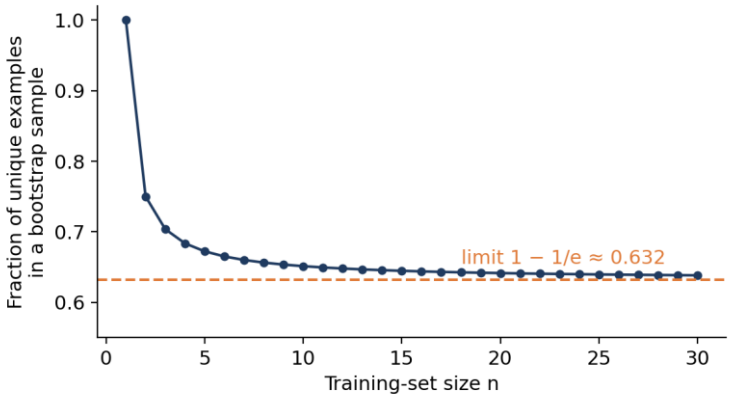


Figure 4. Fraction of unique training examples in a bootstrap sample approaches 0.632.

Worked example

Take a training set of 10 examples numbered 1–10. Three bootstrap samples (drawn with replacement) might look like this:

Round	Bootstrap sample	Distinct examples	Out-of-bag examples
1	5, 10, 7, 4, 4, 8, 8, 10, 8, 9	6	1, 2, 3, 6
2	10, 9, 8, 7, 5, 1, 8, 1, 8, 7	6	2, 3, 4, 6
3	4, 6, 9, 9, 8, 6, 1, 1, 3, 9	6	2, 5, 7, 10

Each sample has the same size as the original data but only 6 distinct examples (close to the expected 63.2 % of 10). Example 2 is out-of-bag in all three rounds, so its OOB prediction would come from the models of rounds 1, 2 and 3 only.

3.2 The algorithm

> **Algorithm: Bagging**
>
> **Input:** training set D of size m; number of models n.
>
> 1. For i = 1 … n: draw a bootstrap sample D_i of size m from D (with replacement).
> 2. Train a base model h_i on D_i.
> 3. **Predict:** classification → $\hat{y}$ = mode{$h_1(x)$, …, $h_n(x)$}; regression → $\hat{y} = (1/n)\ \Sigma\ h_i(x)$.

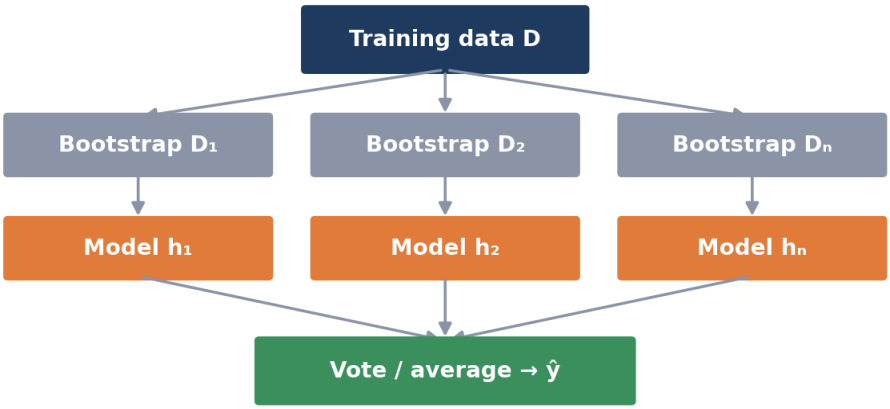


Figure 5. The bagging workflow: bootstrap samples → independent models → vote/average.

3.3 Why bagging reduces variance

If we average *B* models, each with variance σ^2 and average pairwise correlation ρ, the variance of the average is:

$$\mathrm{Var}(\text{average}) = \rho\,\sigma^2 + \frac{1-\rho}{B}\,\sigma^2$$

As B grows, the second term vanishes and only $\rho\sigma^2$ remains. Two lessons follow: (1) adding more trees never hurts and the benefit levels off; (2) **the lower the correlation ρ between trees, the better** – the motivation for random forests. Averaging leaves bias almost unchanged, so bagging is useful for low-bias, high-variance base learners.

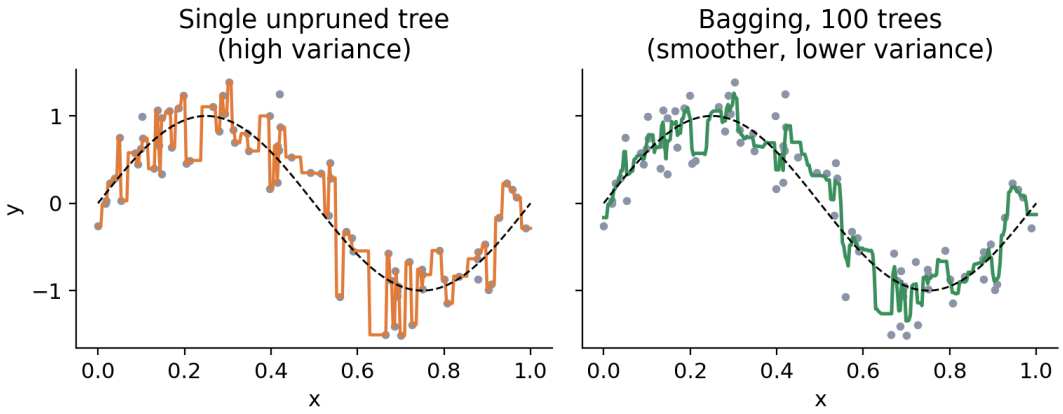


Figure 6. A single unpruned regression tree (left) is jagged; averaging 100 bootstrap trees (right) is much smoother. Dashed line = true function.

3.4 Out-of-bag (OOB) error

Because each tree never sees about a third of the data, we can predict each training example using only the trees for which it was out-of-bag. The resulting **OOB error** is a nearly free, honest estimate of test error – no separate validation set is needed.

3.5 Advantages and limitations

Advantages	Limitations
Reduces variance and overfitting	Does not reduce bias; useless for very simple (high-bias) learners
Trees can be trained in parallel	Less interpretable than a single tree
OOB error gives built-in validation	Uses more memory/time (n models)
Robust to noisy data and outliers	Trees remain correlated if a few features dominate

4. Random Forests

A **random forest** is bagging with decision trees plus one extra source of randomness: at every split, only a random subset of the features is considered. This *decorrelates* the trees (smaller ρ in the variance formula above), so averaging them reduces variance even more than plain bagging. Random forests are among the most widely used algorithms because they perform well "out of the box" with little tuning.

4.1 The algorithm

> **Algorithm: Random Forest**
>
> For b = 1 … B:
>
> 1. Draw a bootstrap sample of the training data.
> 2. Grow a decision tree on it. **At each node**, choose a random subset of features out of the m available, and pick the best split among *only those*.
> 3. Grow the tree deep (little or no pruning).
>
> **Predict:** majority vote (classification) or average (regression) over the B trees.

Choosing the subset size. The number of features per split is a tunable hyperparameter. Common defaults are about √m for classification and m/3 for regression; Breiman also suggested $\log_2(m) + 1$ as a rule of thumb. Smaller subsets give more diverse (less correlated) but individually weaker trees.

4.2 Per tree or per node? (Ho vs. Breiman)

- **Tin Kam Ho (1995)** – the *random subspace method*: each **tree** receives one random subset of features and uses only those throughout.
- **Leo Breiman (2001)** – each **node** of each tree draws a fresh random subset of features, while the tree has access to all features overall. This is the version we mean today by "random forest".

4.3 Why it works: strength and correlation

Breiman showed that the generalisation error of a forest is bounded by:

$$\mathrm{PE} \leq \bar{\rho} \cdot \frac{1 - s^2}{s^2}$$

where $\bar{\rho}$ is the average correlation between trees and **s** is the *strength* of the individual trees (their average margin). Lower correlation → lower error; higher

strength → lower error. Random feature selection slightly lowers each tree's strength but lowers the correlation by more, so the net result is typically better than plain bagging.

4.4 OOB error and feature importance

- **OOB error** works exactly as in bagging and gives a built-in estimate of test error.
- **Feature importance** – although a forest is less interpretable than one tree, we can measure how much each feature reduces impurity (or how much accuracy drops when the feature's values are shuffled). This is covered further in the *Feature Selection* topic.

4.5 Extremely Randomised Trees (ExtraTrees) – for awareness

ExtraTrees go one step further: at each node the split *threshold* is also chosen at random (for each candidate feature), and the best of these random splits is used. No search for the optimal threshold is needed, so training is **faster**, and the extra randomness further **decorrelates** the trees – at the cost of weaker individual trees.

4.6 Key hyperparameters

Hyperparameter	Meaning	Effect of increasing it
Number of trees (B)	Number of trees	Lower variance; more compute; plateaus
Features tried per split (m_try)	Number of random features considered at each split	Higher correlation and strength
Tree depth / minimum leaf size	Tree size limits	Lower variance, higher bias
OOB estimate	Use out-of-bag examples to validate	—

5. Boosting

Boosting converts **weak learners** (models only slightly better than random guessing, e.g. decision *stumps* – trees with one split) into a **strong learner**. Unlike bagging, learners are trained **sequentially**: each new learner concentrates on the examples the earlier ones got wrong. The final prediction is a *weighted* vote.

> **General boosting recipe**
> 1. Start with equal weights on all training examples.
> 2. Train a weak learner on the (re)weighted data.
> 3. Increase the weight of the examples it misclassified (or fit the remaining errors).
> 4. Repeat for many rounds, then combine all learners with a weighted vote: $H(x) = \text{sign}(\Sigma_j \alpha_j h_j(x))$.

The two main families differ in *how* the weights/targets are updated and *how* learners are combined: **adaptive boosting** (AdaBoost) reweights examples; **gradient boosting** (including XGBoost) fits each new learner to the gradient of the loss.

5.1 AdaBoost (Adaptive Boosting)

AdaBoost was proposed by Freund and Schapire (1997). Labels are $y \in \{-1, +1\}$.

> **Algorithm: AdaBoost**
> 1. Initialise weights $w_i = 1/N$ for $i = 1 \dots N$.
> 2. For $t = 1 \dots T$:
> (a) Train weak learner h_t on the training data using weights w.
> (b) Weighted error: $\varepsilon_t = \Sigma\, w_i \cdot 1[h_t(x_i) \neq y_i]$ (if $\varepsilon_t \geq 0.5$, stop).
> (c) Learner weight: $\alpha_t = \tfrac{1}{2} \ln((1 - \varepsilon_t) / \varepsilon_t)$.
> (d) Update: $w_i \leftarrow w_i \cdot \exp(-\alpha_t\, y_i\, h_t(x_i))$, then normalise so $\Sigma\, w_i = 1$.
> 3. **Final classifier:** $H(x) = \text{sign}(\Sigma_t \alpha_t h_t(x))$.

$$\alpha_t = \frac{1}{2} \ln \left(\frac{1 - \varepsilon_t}{\varepsilon_t} \right)$$

$$w_i \leftarrow w_i \exp \left(-\alpha_t\, y_i\, h_t(x_i) \right)$$

$$H(x) = \text{sign}\left[\sum_{t=1}^{T} \alpha_t \, h_t(x)\right]$$

Reading the formulas. α_t is large when the error ε_t is small, so accurate learners get a louder vote. In step (d), a misclassified example has $y_i h_t(x_i) = -1$, so its weight is multiplied by $\exp(+\alpha_t)$, which is greater than 1; a correctly classified one is multiplied by $\exp(-\alpha_t)$, which is less than 1.

Worked example (one round)

Ten training examples start with weight 0.1 each. The first stump misclassifies 3 of them.

Quantity	Value
Weighted error ε_1	$3 \times 0.1 = 0.30$
Learner weight α_1	$\tfrac{1}{2} \ln(0.70 / 0.30) = 0.4236$
Misclassified: unnormalised weight	$0.1 \times \exp(0.4236) = 0.1528$
Correct: unnormalised weight	$0.1 \times \exp(-0.4236) = 0.0655$
Sum of unnormalised weights	$3(0.1528) + 7(0.0655) = 0.9165$
New normalised weights	Misclassified: **0.1667** each (total 0.50); correct: **0.0714** each (total 0.50)

After the update the 3 hard examples carry **half** of the total weight, so the next stump is forced to concentrate on them. Figure 7 shows three rounds on a toy dataset – marker size is proportional to the example's weight.

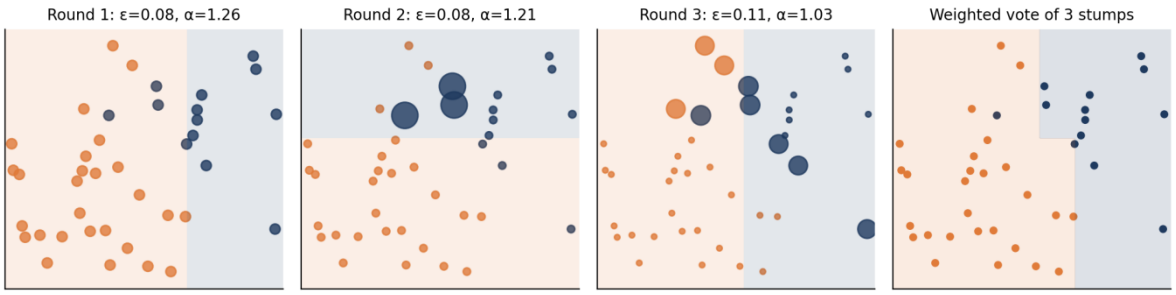


Figure 7. AdaBoost on a toy dataset: three weighted stumps (left three panels) and their weighted vote (right).

Properties of AdaBoost

- Typically uses stumps or shallow trees; each is a high-bias, low-variance learner, so boosting mainly **reduces bias**.

- Often resists overfitting for many rounds, but is **sensitive to noisy labels and outliers**, because misclassified noisy points keep gaining weight.
- Number of rounds T and the learning rate are the key hyperparameters; use cross-validation to choose them.

AdaBoost: advantages	AdaBoost: limitations
Simple; few hyperparameters	Sensitive to noisy labels and outliers
Often high accuracy with stumps	Sequential, so cannot be parallelised across rounds
Automatic focus on hard examples	Needs weak learners that beat 50 % accuracy

5.2 Gradient Boosting

Gradient boosting (Friedman, 2001) treats boosting as **gradient descent in function space**. We build an additive model $F(x) = F_0 + \nu \Sigma\, h_m(x)$, where each new tree h_m is fitted to the **negative gradient of the loss** with respect to the current predictions (called *pseudo-residuals*).

> **Algorithm: Gradient Boosting**
>
> 1. Initialise with a constant: $F_0(x) = \arg\min_c \Sigma\, L(y_i, c)$ (the mean of y for squared error).
> 2. For m = 1 … M:
>
> (a) Compute pseudo-residuals: $r_i = -\partial L(y_i, F(x_i)) / \partial F(x_i)$ evaluated at $F = F_{m-1}$.
>
> (b) Fit a small regression tree h_m to the targets r_i.
>
> (c) Update: $F_m(x) = F_{m-1}(x) + \nu \cdot h_m(x)$, with learning rate (shrinkage) $0 < \nu \leq 1$.
>
> 3. Output F_M(x).

$$r_i = -\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)}$$

$$F_m(x) = F_{m-1}(x) + \nu h_m(x)$$

Special case – squared error. For $L = \tfrac{1}{2}(y - F)^2$ the negative gradient is simply $y - F$, i.e. the ordinary **residual**. So each tree is fitted to what the model currently gets wrong, and adding it shrinks the residual. For classification we use the log-loss and the residuals become (y − predicted probability).

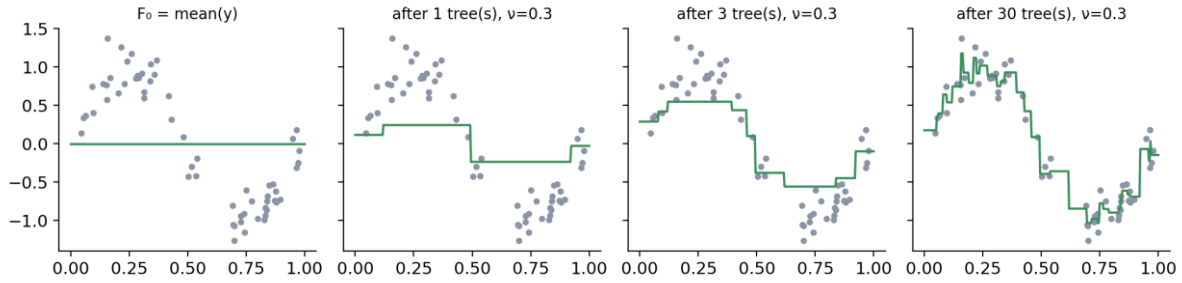


Figure 8. Gradient boosting for regression: starting from the mean, each depth-2 tree is fitted to the current residuals and added with learning rate v = 0.3.

- **Learning rate v:** small values need more trees but generalise better. A standard trade-off is *small v + large M*.
- **Tree depth:** shallow trees (depth 3–6) capture interactions without high variance.
- **Subsampling** (stochastic gradient boosting): fit each tree on a random fraction of the rows to add randomness and reduce variance.

5.3 XGBoost (Extreme Gradient Boosting)

XGBoost (Chen & Guestrin, 2016) is a highly optimised, regularised implementation of gradient boosted trees. It became popular because it is fast, accurate and handles real-world data issues. What it adds to the basic algorithm falls into two groups.

A. Algorithmic improvements

Regularised objective. At round t, XGBoost minimises

$$\text{Obj} = \sum_i L(y_i, \hat{y}_i) + \sum_k \Omega(f_k)$$

$$\Omega(f) = \gamma T + \frac{1}{2}\lambda \sum_{j=1}^{T} w_j^2$$

where T is the number of leaves of the tree and w_j are the leaf scores. γ penalises extra leaves and λ shrinks leaf scores (L2). This controls model complexity directly, which plain gradient boosting does not.

Second-order approximation. The loss is expanded to second order around the current prediction using the gradient g_i and the Hessian h_i of the loss for each example. For a leaf containing the set of examples I, with $G = \Sigma g_i$ and $H = \Sigma h_i$:

$$w^* = -\frac{G}{H+\lambda}$$

Split gain. To decide whether to split a node into left (L) and right (R) children, XGBoost computes:

$$\text{Gain} = \frac{1}{2}\left[\frac{G_L^2}{H_L+\lambda} + \frac{G_R^2}{H_R+\lambda} - \frac{(G_L+G_R)^2}{H_L+H_R+\lambda}\right] - \gamma$$

A split is made only if the gain is positive, so γ acts as a built-in pruning threshold. Using Hessians as well as gradients gives better-informed steps (similar in spirit to Newton's method).

B. Practical / engineering features

- **Sparsity-aware split finding** – learns a default direction for missing values, so missing data are handled natively.
- **Shrinkage and column subsampling** (as in random forests) to reduce overfitting.
- **Efficient split search** – pre-sorted/histogram-based approximate splits (weighted quantile sketch) for large data.
- **Parallel, cache-aware computation** and out-of-core learning; GPU support.
- **Built-in cross-validation and early stopping** on a validation set.

Key XGBoost hyperparameters

Parameter	Role	Typical guidance
Number of rounds (T)	Number of boosting rounds	Large, with early stopping
Learning rate (η)	Shrinkage on each tree	0.01–0.3; smaller = safer, needs more trees
Maximum tree depth	Tree depth	3–8; deeper = higher variance
Row and column subsampling	Row / column sampling	0.5–1.0; <1 reduces overfitting
Regularisation λ and γ	L2 penalty; minimum gain to split	Increase to regularise more
Minimum child weight	Minimum Hessian sum in a leaf	Increase to avoid tiny leaves

XGBoost vs. standard gradient boosting

Aspect	Standard gradient boosting	XGBoost
Loss approximation	First order (gradient only)	Second order (gradient and Hessian)
Regularisation	Only shrinkage / depth limits	Explicit penalties γ and λ in the objective
Missing values	Must be imputed	Learned default direction per split
Pruning	Stops by depth	Grows then prunes using gain $-\gamma$
Speed	Single-threaded, exact splits	Parallel, histogram / approximate splits

XGBoost: advantages	XGBoost: limitations
State-of-the-art accuracy on tabular data	Many hyperparameters to tune
Fast and scalable; handles missing data	Less interpretable than a single tree
Built-in regularisation and early stopping	Can still overfit small or noisy data

6. Comparison and Choosing a Method

6.1 Bagging vs. boosting

Aspect	Bagging / Random Forest	Boosting (AdaBoost, GB, XGBoost)
Training	Parallel, independent models	Sequential; each depends on the previous
Base learner	Strong, high-variance (deep trees)	Weak, high-bias (stumps, shallow trees)
Data per model	Bootstrap sample	Reweighted data / residuals
Main effect	Reduces **variance**	Reduces **bias** (and some variance)
Combination	Equal vote / average	Weighted sum
Noise sensitivity	Robust	Can overfit noisy data
Tuning effort	Low	Higher (learning rate, depth, rounds)
Typical accuracy	Very good	Often the best on tabular data when tuned

6.2 Method-by-method summary

Method	Core idea	Key formula / feature	Watch out for
Bagging	Average models on bootstrap samples	63.2 % unique per sample; OOB error	No bias reduction
Random Forest	Bagging + random feature subset per node	$PE \le \bar{\rho}(1 - s^2)/s^2$	Large forests can be slow
AdaBoost	Reweight misclassified examples	$\alpha = \frac{1}{2} \ln((1 - \varepsilon)/\varepsilon)$	Noisy labels, outliers
Gradient Boosting	Fit residuals (negative gradient)	$F \leftarrow F + \nu \cdot h$	Needs tuning; sequential
XGBoost	Regularised, 2nd-order gradient boosting	$w^* = -G/(H+\lambda)$; gain with γ	Many hyperparameters